```
import keras
```

```
import ssl
import urllib.request

# Work around local CA certificate issues when keras downloads CIFA
ssl_context = ssl._create_unverified_context()
ssl._create_default_https_context = ssl._create_unverified_context
urllib.request.install_opener(
    urllib.request.build_opener(urllib.request.HTTPSHandler(context
 )

import numpy as np

(x_train, y_train), (x_test, y_test) = keras.datasets.cifar10.load_

np.savez("cifar10_train.npz", x_train=x_train, y_train=y_train)
np.savez("cifar10_test.npz", x_test=x_test, y_test=y_test)
```

```
# Train shape: (Amount of pictures, size height, size width, colour
# Test shape: (Amount of pictures, label)
print(x_train.shape)
print(y_train.shape)
```

```
(50000, 32, 32, 3)
(50000, 1)
```

```
# Normalise images, because the RBG-image values are between 0 and
x_train = x_train / 255
x_test = x_test / 255
```

```
print(y_train[1])

import matplotlib.pyplot as plt
print(y_train[1])

plt.figure(figsize=(2,2))
plt.imshow(x_train[1])
plt.axis('off')
plt.show()
```

[9]
[9]



```
# Keep labels numeric for training, and map them only when you want
convert = {0: "airplane",
           1:"automobile",
           2:"bird",
           3: "cat",
           4: "deer",
           5: "dog",
           6: "frog",
           7: "horse",
           8: "ship",
           9: "truck"}

inverse_convert = {name: index for index, name in convert.items()}

if isinstance(y_train[0], str):
    y_train = np.array([inverse_convert[label] for label in y_train
    y_test = np.array([inverse_convert[label] for label in y_test],
else:
    y_train = np.array([label[0] for label in y_train], dtype="int6
    y_test = np.array([label[0] for label in y_test], dtype="int64"
```

```
y_train = keras.utils.to_categorical(y_train, 10)
y_test = keras.utils.to_categorical(y_test, 10)
```

```
model = keras.Sequential()
model.add(keras.layers.Conv2D(filters=32, kernel_size=(3, 3), activ
model.add(keras.layers.Conv2D(filters=32, kernel_size=(3, 3), activ
model.add(keras.layers.MaxPool2D(pool_size=(2, 2)))

# add dropout 1:
model.add(keras.layers.Dropout(rate=0.25))

# add repeated layers
model.add(keras.layers.Conv2D(filters=32, kernel_size=(3, 3), activ
model.add(keras.layers.Conv2D(filters=32, kernel_size=(3, 3), activ
model.add(keras.layers.MaxPool2D(pool_size=(2, 2)))
model.add(keras.layers.Dropout(rate=0.25))

model.add(keras.layers.Flatten())
model.add(keras.layers.Dense(512, activation="relu"))

# add dropout 2:
model.add(keras.layers.Dropout(rate=0.50))

model.add(keras.layers.Dense(10, activation="softmax"))


optimizer = keras.optimizers.RMSprop(learning_rate=0.0001, weight_d
model.compile(loss="categorical_crossentropy", optimizer=optimizer,
```

```
history = model.fit(x_train, y_train, batch_size=32, epochs=20, ver
```

```
Epoch 1/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 19s 12ms/step - accuracy: 0.3022 - lo
Epoch 2/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 18s 11ms/step - accuracy: 0.4095 - lo
Epoch 3/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 17s 11ms/step - accuracy: 0.4683 - lo
Epoch 4/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 18s 11ms/step - accuracy: 0.5128 - lo
Epoch 5/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 18s 11ms/step - accuracy: 0.5420 - lo
Epoch 6/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 20s 13ms/step - accuracy: 0.5623 - lo
Epoch 7/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 19s 12ms/step - accuracy: 0.5799 - lo
Epoch 8/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 18s 12ms/step - accuracy: 0.5962 - lo
Epoch 9/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 19s 12ms/step - accuracy: 0.6115 - lo
Epoch 10/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 17s 11ms/step - accuracy: 0.6211 - lo
Epoch 11/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 17s 11ms/step - accuracy: 0.6336 - lo
Epoch 12/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 18s 11ms/step - accuracy: 0.6440 - lo
Epoch 13/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 18s 11ms/step - accuracy: 0.6497 - lo
Epoch 14/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 18s 11ms/step - accuracy: 0.6597 - lo
Epoch 15/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 18s 11ms/step - accuracy: 0.6663 - lo
Epoch 16/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 18s 11ms/step - accuracy: 0.6722 - lo
Epoch 17/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 18s 11ms/step - accuracy: 0.6805 - lo
Epoch 18/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 18s 11ms/step - accuracy: 0.6862 - lo
Epoch 19/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 18s 11ms/step - accuracy: 0.6920 - lo
Epoch 20/20
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 18s 11ms/step - accuracy: 0.6975 - lo
```

Question 1

Overfitting is a phenomena where the model is too complex for the data it is trying to represent. Instead of capturing the pattern in the data, the model is so sophisticated that it starts to model the noise in the data as well, called overfitting. In such cases, the model has captured the training data almost perfectly, while not generalizing well to other data. This is observed by a training error that keeps decreasing while the error for unseen cases doesn't follow. Exact figures are difficult but is best detected as a growing gap between the training and validattion curves.

The following citation explains dropout very well: "Overfitting can be reduced by using “dropout” to prevent complex co-adaptations on the training data. On each presentation of each training case, each hidden unit is randomly omitted from the network with a probability of 0.5, so a hidden unit cannot rely on other hidden units being present." [2] That is, dropout randomly sets some proportion of unit outputs to zero during training, with a new random mask each training iteration [1]. Then at test time the 'mean' network is used with all outgoing weights weighted by the proportion of times they were inactive (0.5 from the citation from before).

Neurons in the network now cannot rely on other units in the network so they must all be optimised on their own, greatly benefitting the model. It also causes the network to act similarly to many smaller networks that are combined, reducing complexity of the model thus also reducing overfitting.

References

- [1] Keras `Dropout` layer docs: https://keras.io/api/layers/regularization_layers/dropout/
- [2] Hinton, G. E., Srivastava, N., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. R. (2012). Improving neural networks by preventing co-adaptation of feature detectors. https://arxiv.org/pdf/1207.0580

AI usage

I used ChatGPT to find the paper introducing Dropout since I only found docs from Keras (useful) or blog posts (not so useful) via Google. I was already familiar with Dropout so I used the paper to fact-check my own understanding of the concept.

Question 2:

Plot is included below

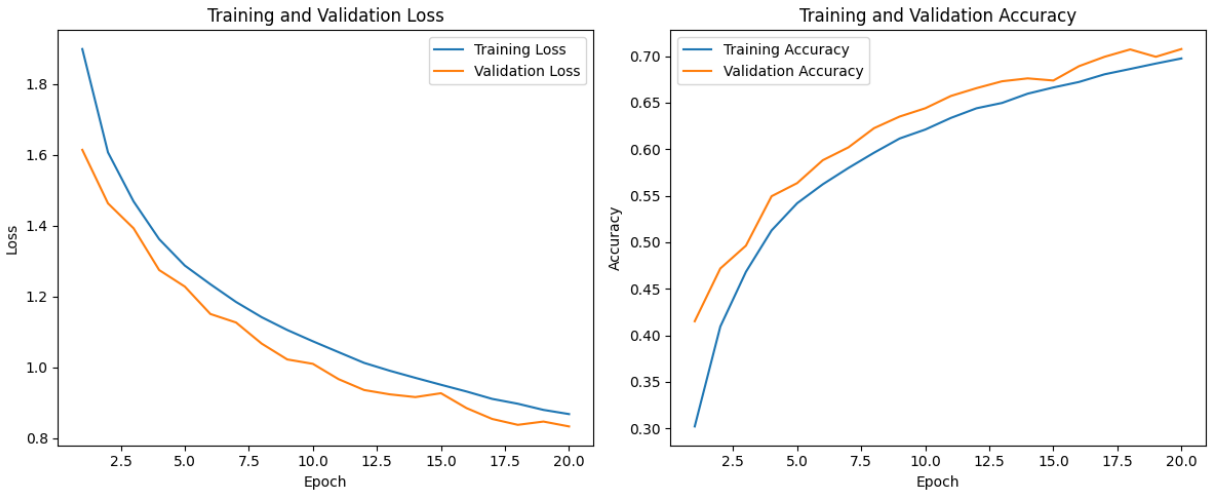
```
epochs = range(1, len(history.history['loss']) + 1)

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1) # 1 row, 2 columns, first plot
plt.plot(epochs, history.history['loss'], label='Training Loss')
plt.plot(epochs, history.history['val_loss'], label='Validation Los
plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.subplot(1, 2, 2) # 1 row, 2 columns, second plot
plt.plot(epochs, history.history['accuracy'], label='Training Accur
plt.plot(epochs, history.history['val_accuracy'], label='Validation
plt.title('Training and Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.tight_layout()
plt.show()
```



Question 3:

The logs show the following: The model from the learning based assignment (model1) took about 14s/epoch (375 steps $\times$ ~37-38ms). The model above (model2) took about 18s/epoch (1563 steps $\times$ ~11-12 ms/step). So model2 is $\frac{18}{14} \approx$ $1.3\times$ slower per epoch.

Over full training, that is 360s for 20 epochs for model2 vs 84s for 6 epochs for model1. Total training thus slowed by $\frac{360}{84} \approx$ $4.3\times$.

Question 4:

1. As shown in the computations, the epochs of the former model consisted of 375 steps and the latter model 1563 steps to complete one epoch. That said, the second model takes batches of size 32 while the first one took batches of 128. Each step has fixed costs so more steps are therefore inefficient. Even though the second model has batches that are 4$\times$ as big, it also has 4$\times$ as many steps.
2. The CIFAR model has 4 layers and the MNIST model 2 convolutional layers, it also has extra dropout and pooling layers. So each each forsward pass of the model needs to perform more operations.

Question 5:

1. The performance: The MNIST CNN ends at 98.7% validation accuracy (98.6% test) with a validation loss of 0.05. The CIFAR-10 CNN ends at 70.7% validation accuracy with a validation loss of 0.83. CIFAR-10 contains much more difficult images to categorise so this difference is not that strange, even with the more complex model. Objects vary in position, scale, viewpoint,

colour and background, and some classes look alike (cat/dog, automobile/truck). The same digits in the MNIST dataset are much more alike to each other, as they have the same position, are centred, have the same size and are grey/white on a black background.

2. Speed of learning: The MNIST model of question 6 reaches 97.6% validation accuracy after one epoch, and its curve of the validation accuracy stays relatively flat after three epochs. The CIFAR-10 model starts at 42% validation accuracy and climbs slowly to 71% over 20 epochs. One reason for slower learning could be that the difference in complexity of the task, the digits are simpler and could therefore be learned after one pass of the data (one epoch), while objects are more difficult to learn and extract the features for. A second reason for the speed of learning is the optimiser settings, as the MNIST model used Adadelta with learning rate 1.0. The CIFAR-10 model used RMSprop with learning rate 0.0001, therefore each update is much smaller.
3. Convergence: The MNIST model has converged in 6 epochs: its validation loss plateaus from epoch 4 onwards. The CIFAR-10 model has not converged after 20 epochs. Both of its losses are still falling and both accuracies are still rising, so more epochs would very likely improve it.
4. Validation > training: In both models, validation accuracy is higher than the training accuracy and validation loss is lower than the training loss. The cause is that Dropout is active during training and switched off during evaluation, so the training scores come from an incomplete network which are averaged while the network is still improving. The validation scores are measured at the end of the epoch.
5. Smoothness: The CIFAR curves are smooth compared to the graph of model4 in the learning based assignment. This is likely due to the smaller learning rate of the newer model.

Question 6:

1. As mentioned earlier, MNIST digits are always centered and with (almost) fixed sizes so each pixel can receive its own weight to count towards a certain number. In de CIFAR data, images are different sizes, positions etc so the network has to learn objects in different configurations.
2. CIFAR is in colour (3 dimensions) while MNIST is gray-scale (1 dimension). These extra inputs increase the risk of overfitting for the linear model when

handling 50.000 images.

3. CIFAR items of different classes share similar features, e.g. cats and dogs share features but are differnt classes. Thus, the model needs to be able to seperate these features per class which in turn needs hierarchal features which cannot be distinguished by a dense layer.

Double-click (or enter) to edit